

SmartFit Trainer

An IoT-Based Wrist-Worn Fitness Monitor

Group B

July 2, 2026

1 Team Members

Name	Email
Rionela Kovaci	rionela.kovaci@stud.hshl.de
Christian Percival	christian.percival@stud.hshl.de
Mohamed Abdo	mohamed-sayed-mohamed.abdo@stud.hshl.de

1 Introduction

The **Internet of Things (IoT)** refers to a network of physical devices connected to the internet, capable of collecting and exchanging data without human intervention. In fitness applications, IoT enables real-time monitoring of vital signs and physical activity, making health tracking accessible and continuous.

A **Wireless Sensor Network (WSN)** is a collection of spatially distributed sensors that communicate wirelessly to monitor physical or environmental conditions. A typical WSN is organised into three conceptual layers: a *perception layer* that senses the physical world, a *network layer* that transports the sensed data, and an *application layer* that consumes and presents the data to an end user. In this project the ESP32 acts as the sensor node (perception layer), the Raspberry Pi acts as the gateway and MQTT broker (network layer), and the web application acts as the monitoring interface (application layer).

The target application is **SmartFit Trainer**. A wrist-worn fitness monitor that tracks heart rate and step count in real time, providing immediate feedback through visual and audio alerts, and displaying all data live on a web application. The device is intended for athletes who want a low-cost, self-hosted alternative to commercial fitness trackers, with full transparency over how the data is measured, transmitted, and displayed. This report documents the full development cycle, covering the system concept, the technologies used, implementation details, and verification results obtained from component-level, integration-level, and full-system testing.

2 Concept Description

2.1 Block Diagram

The block definition diagram in Fig. 1 shows the complete system architecture. Sensors and a buzzer actuator connect to the Arduino Nano ESP32, which processes data and publishes it over Wi-Fi to the Raspberry Pi MQTT broker. The web application subscribes to the broker and displays live data to the athlete. Each connection is typed with the signal or payload it carries: analog and digital flow properties between the ESP32 and its sensors, and an MQTT payload flow between the ESP32, the broker, and the web application.

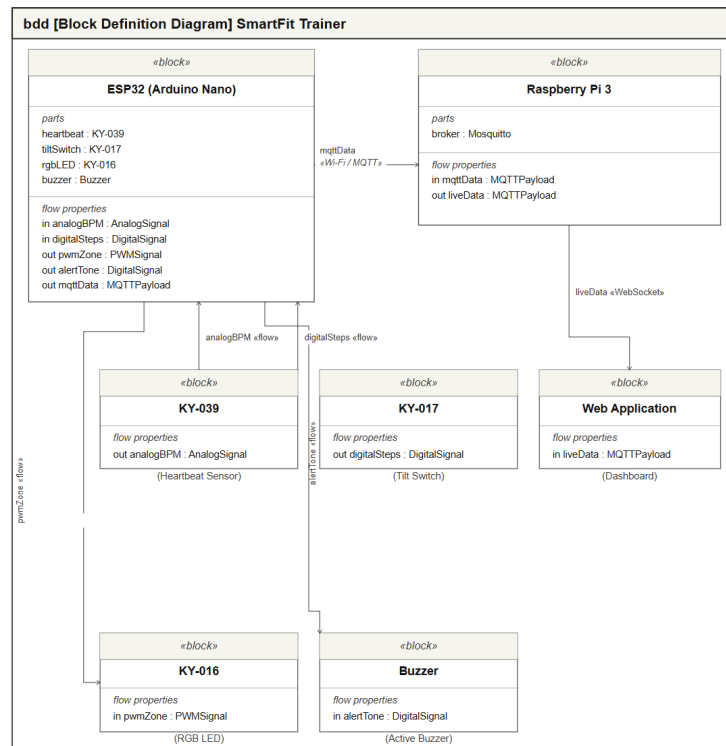


Figure 1: SmartFit Trainer block definition diagram (BDD)

2.2 Devices, Sensors and Actuators

Component	Description
Arduino Nano ESP32	Main MCU; reads sensors and publishes data via MQTT over Wi-Fi
KY-039 Heartbeat Sensor	IR-based analog heart rate sensor; signal on pin A0
KY-017 Tilt Switch	Detects arm swing for step counting; connected to D5
KY-016 RGB LED Module	Common-cathode RGB LED; PWM on D10/D11/D12 for zone display
Active Buzzer (KY-012)	Self-contained piezo actuator for danger-zone alerts; pin D6
Raspberry Pi 3 Model B	Runs Mosquitto MQTT broker; WSN gateway
Web Application	Live dashboard: BPM, zone, steps, and alerts

2.3 Main Application

The athlete wears the device on the wrist during exercise. The device continuously reads heart rate, detects the training zone, drives the RGB LED and buzzer accordingly, and streams all data wirelessly to the web app for live monitoring. No buttons need to be pressed and no companion app needs to be paired, the device begins publishing as soon as it connects to Wi-Fi, and the dashboard begins displaying as soon as it connects to the broker.

3 Project and Team Management

3.1 Project Method

The project followed an **incremental development approach** structured into five stages: (1) hardware setup and sensor testing, (2) MQTT broker configuration on the Raspberry Pi, (3) ESP32 firmware development, (4) web application development, and (5) full system integration and testing. Each stage was completed and verified before advancing to the next, following a verification-first policy: a sensor was never wired into the full firmware until it had already been

proven correct in isolation with a standalone test sketch (see Section 6). All source files were tracked in a shared Git repository throughout development.

3.2 Task Distribution

Task	Details	Member
Raspberry Pi setup	OS install, SSH config, Mosquitto install & config	All
Hardware wiring	Breadboard layout, sensor connections, pin assignment	All
ESP32 firmware	Wi-Fi, MQTT client, BPM algorithm, zone logic, actuators	All
System diagrams	BDD, use-case, sequence, and state machine diagrams	All
Web application	HTML/CSS/JS dashboard, MQTT.js WebSocket integration	All
Full system testing	End-to-end integration test and debugging	All
Documentation	LaTeX report writing, figures, references	All

4 Technologies

4.1 Sensors and Actuators

The **KY-039** heartbeat sensor uses an infrared LED and photodetector to measure blood flow through the fingertip. It outputs an analog signal read on pin A0. A peak detection algorithm with a minimum 400 ms inter-beat interval calculates BPM and filters motion noise, tracking a decaying running maximum so the detector adapts to slow baseline drift while still reacting quickly to a genuine pulse.

The **KY-017** tilt switch detects arm swing for step counting via a digital signal on pin D5; only a falling edge (HIGH→LOW) is treated as a step event, avoiding double-counting a single swing. The **KY-016** RGB LED module provides PWM zone indication on D10/D11/D12, giving five visually distinct colours across the heart-rate zones (Section 5.5).

The **KY-012** active buzzer is a self-contained piezo actuator: driving its signal pin HIGH produces a fixed-frequency tone with no waveform generation required — a 150 ms pulse for zone 4 and a sustained tone for zone 5.

4.2 Communication Protocol

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe protocol for IoT, chosen over direct HTTP polling because it decouples the ESP32 from the web application entirely. the firmware never needs to know how many dashboards are listening. The ESP32 publishes sensor data to four topics; the Mosquitto broker on the Raspberry Pi forwards messages; the web application subscribes via WebSocket on port 9001, since browsers cannot speak MQTT over its native TCP port 1883 directly.

MQTT Topic	Payload
smartfit/vitals/bpm	Heart rate in BPM
smartfit/vitals/zone	Current HR zone (1–5)
smartfit/motion/steps	Cumulative step count
smartfit/alerts/danger	Danger flag: 1 for zones 4–5, 0 otherwise

4.3 Programming Languages

ESP32 firmware is written in **C++** using the Arduino framework and IDE. The Raspberry Pi is configured via **Bash** over SSH. The web application uses **HTML**, **CSS**, and **JavaScript**

with the MQTT.js library, which implements MQTT over WebSocket in the browser without any server-side component. the dashboard is a single static HTML file.

5 Implementation

5.1 System Architecture

The system follows a three-layer WSN architecture. The **Perception Layer** consists of the ESP32 reading all sensors every 60 ms. The **Network Layer** handles MQTT communication between the ESP32 and the Raspberry Pi broker. The **Application Layer** is the web app displaying all live data. Each layer can be tested and replaced independently, as long as it continues to speak the MQTT topic contract in Section 4.2.

5.2 Raspberry Pi Setup

The Raspberry Pi 3 was configured with Raspberry Pi OS 64-bit via SSH. Mosquitto was installed and configured to accept TCP connections on port 1883 and WebSocket connections on port 9001:

```
sudo apt install mosquitto mosquitto-clients -y
sudo systemctl enable mosquitto && sudo systemctl start mosquitto
listener 1883
allow_anonymous true
listener 9001
protocol websockets
```

Anonymous access was enabled for the scope of this lab project, since the broker is only reachable on a local Wi-Fi hotspot created for testing.

5.3 Hardware Wiring

Component	Wiring
KY-039 Heartbeat	Signal → A0, VCC → 3V3, GND → GND
KY-017 Tilt	Signal → D5, VCC → 3V3, GND → GND
KY-016 RGB LED	Red → D12, Green → D11, Blue → D10, GND → GND
Active Buzzer (KY-012)	Signal → D6, VCC → 3V3, GND → GND

5.4 ESP32 Firmware

The firmware connects to Wi-Fi on startup, then establishes an MQTT session with the broker. The main loop runs every 60 ms: reads the heartbeat sensor (peak detection), reads the tilt switch, determines the HR zone, drives the LED and buzzer, and publishes all four MQTT topics. A non-blocking reconnect mechanism retries every 5 s on connection loss, so a temporary broker outage does not stall sensor sampling.

5.4.1 BPM Calculation

Each detected beat updates a rolling inter-beat interval. Given the interval Δt in milliseconds between two consecutive beats, the instantaneous heart rate is

$$\text{BPM} = \frac{60000}{\Delta t}.$$

To suppress noise, the firmware averages the last five instantaneous BPM readings via a circular buffer, rejecting intervals outside 350–2000 ms as sensor artefacts before they enter the buffer.

5.5 Heart Rate Zone Logic

Zone	BPM Range	LED	Buzzer	App Alert
1	< 100	Blue	Off	None
2	100–120	Green	Off	None
3	120–140	Yellow	Off	None
4	140–160	Orange	Single beep	Warning
5	> 160	Red	Continuous alarm	Danger

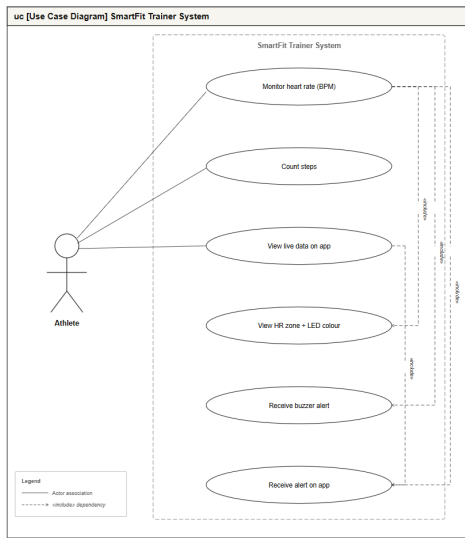
Zone boundaries loosely follow standard percentage-of-maximum heart-rate training zones, simplified to fixed BPM thresholds. Every zone transition publishes an updated `smartfit/vitals/zone` value, and zones 4 and 5 additionally drive the `smartfit/alerts/danger` flag.

5.6 Web Application

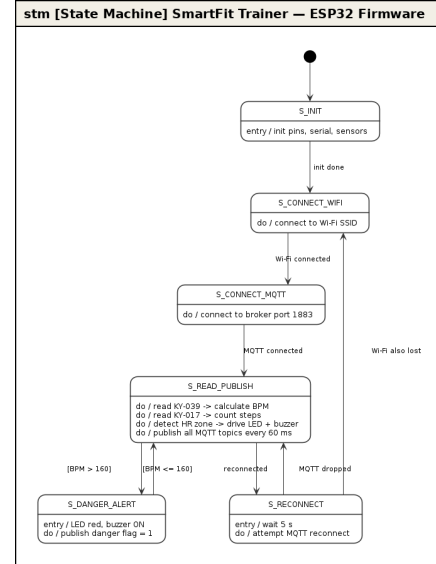
The web dashboard is a single static HTML page that connects to the Raspberry Pi broker over WebSocket using MQTT.js and subscribes to the `smartfit/#` wildcard topic. Incoming BPM and step values update two numeric readouts; zone values update a colour-coded badge; the danger flag toggles a persistent alert banner, fully event-driven, no page reload required.

5.7 UML Diagrams

The use case diagram (Fig. 2a) and state machine diagram (Fig. 2b) show the athlete's interactions and the internal firmware states of the ESP32. The sequence diagram (Fig. 3) captures the full event timeline during a workout session from sensor acquisition through MQTT transmission to web display and zone alerts.



(a) Use case diagram



(b) ESP32 state machine diagram

Figure 2: Use case and state machine diagrams for SmartFit Trainer

The state machine in Fig. 2b shows that an MQTT drop routes into a dedicated reconnect state without interrupting sensor sampling, while only a full Wi-Fi loss forces a return to the initial connection sequence.

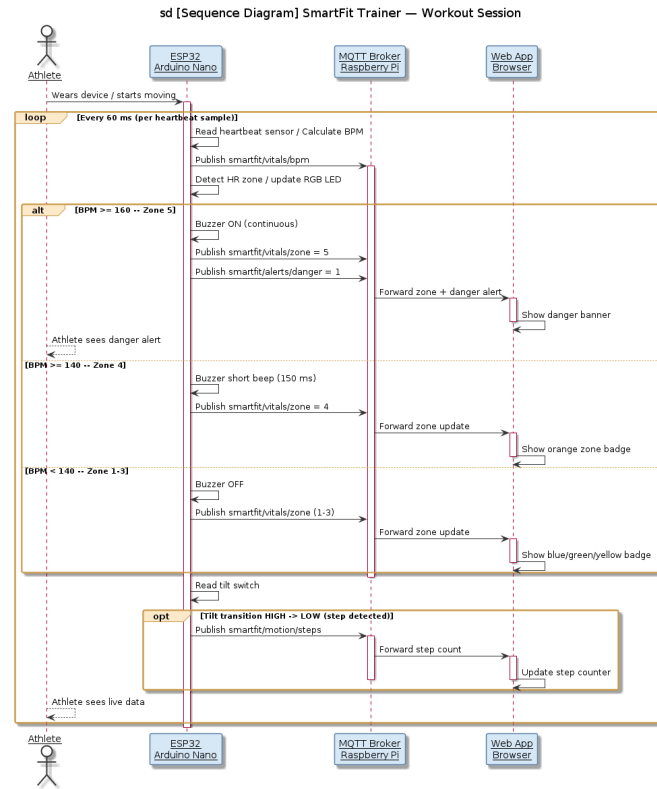


Figure 3: SmartFit Trainer sequence diagram — sensor read through web display

6 Results

6.1 Component-Level Testing

Each hardware component was tested individually using dedicated standalone sketches before firmware integration. The test procedure, expected output, observed result, and pass/fail status are summarised in Table 1.

Component	Test Procedure	Expected / Observed	Result
KY-039 Heartbeat	Fingertip placed on sensor; analog value plotted on Serial Plotter at 9600 baud	Clear sinusoidal pulse waveform rising and falling with each heartbeat; peaks separated by ≥ 400 ms	Pass
KY-017 Tilt Switch	Sensor tilted manually; digital state read on D5 and printed to Serial Monitor	Tilt detected! printed on each arm-swing tilt; no false triggers at rest	Pass
KY-016 RGB LED	Firmware cycled Red→Green→Blue at 1 s intervals via PWM on D10/D11/D12	Each colour channel illuminated distinctly and in correct sequence	Pass
KY-012 Active Buzzer	Signal pin toggled HIGH for 4 s and LOW for 2 s in loop	Audible fixed-frequency tone on HIGH; silence on LOW; no PWM needed	Pass

Table 1: Component-level test results

6.2 MQTT Integration Testing

After individual component verification, the ESP32 firmware was extended with Wi-Fi and MQTT connectivity. Data flow was validated on the Raspberry Pi using `mosquitto_sub -t smartfit/#` to subscribe to all topics simultaneously. BPM values were confirmed to update within the 60 ms loop period, zone transitions triggered correct LED colour changes, and step increments were reflected in the `smartfit/motion/steps` topic without packet loss.

Fig. 4a confirms the Mosquitto broker service was active and listening on ports 1883 and 9001. Fig. 4b shows the live terminal output of `mosquitto_sub`, confirming all four topics were publishing correctly in real time.

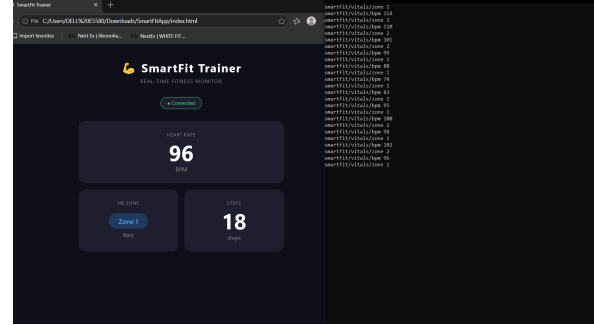
```

pi@raspberrypi2:~$ sudo systemctl status mosquitto
mosquitto.service - Mosquitto MQTT Broker
   Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-06-15 13:31:19 CEST; 1h 10min ago
   Invocation: c9a3e23ae2458fa5d87d2d56420
   Main PID: 1785 (mosquitto)
   Tasks: 1 (limit: 8742)
   CPU: 2.47s
   Group: /system.slice/mosquitto.service
          └─1785 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

Jun 15 13:31:19 raspberrypi2 systemd[1]: Starting mosquitto.service - Mosquitto MQTT Broker...
Jun 15 13:31:19 raspberrypi2 systemd[1]: Started mosquitto.service - Mosquitto MQTT Broker.

```

(a) Mosquitto broker active on Raspberry Pi



(b) Live MQTT data confirmed via `mosquitto_sub`

Figure 4: MQTT broker status and live topic data flow

6.3 Full System Verification

A full end-to-end test was performed with all components active simultaneously. Table 2 documents each functional requirement, the test method applied, and the verified outcome.

Requirement	Test Method & Observed Outcome	Result
Wi-Fi connection on boot	Power-cycled ESP32; Serial Monitor confirmed WiFi connected within 5 s	Pass
MQTT broker connection	Session confirmed via Connected to broker log and Mosquitto client list	Pass
BPM measurement	Fingertip on KY-039; stable BPM in range (60–100) on Serial Monitor and web app	Pass
Step counting	Arm swings performed; <code>steps</code> topic incremented on each tilt event	Pass
Zone 1 — LED blue (<100 BPM)	BPM held below threshold; blue channel on KY-016 confirmed visually	Pass
Zone 4 — orange + beep (140–160 BPM)	Firmware override to zone 4; orange LED and 150 ms buzzer pulse confirmed	Pass
Zone 5 — red + alarm (>160 BPM)	Firmware override to zone 5; red LED, sustained tone, <code>danger = 1</code> published	Pass
Web app live display	BPM, zone, and step count updated within 1 s of sensor change in browser	Pass
Web app danger alert	Zone 5 triggered; red alert banner appeared within one MQTT message cycle	Pass
MQTT auto-reconnect	Broker restarted mid-session; ESP32 reconnected within 5 s automatically	Pass

Table 2: Full system functional verification — 10/10 passed

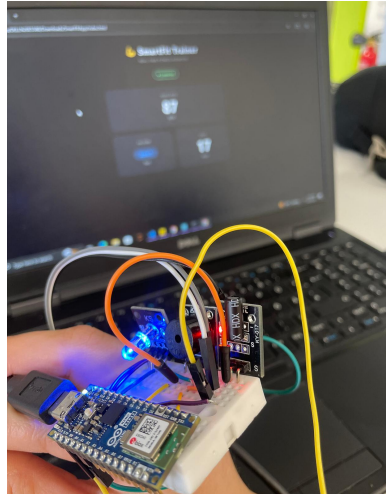


Figure 5: Complete SmartFit Trainer system in operation — hardware and web application

6.4 Challenges

All ten functional requirements passed verification. The main technical challenges encountered and resolved were: (1) Mosquitto WebSocket configuration requiring an explicit `protocol websockets` directive on port 9001; (2) ESP32 upload failures caused by USB driver instability, resolved by holding the BOOT button during flashing; and (3) heartbeat sensor sensitivity to ambient light, mitigated by shielding the KY-039 and tuning the peak-detection threshold.

The largest source of debugging time was not firmware logic itself but confirming which network layer a fault sat in — Wi-Fi, MQTT, or the browser's WebSocket session. Subscribing to `smartfit/#` directly on the Raspberry Pi with `mosquitto_sub` early in integration testing made it possible to isolate broker-side issues from firmware-side issues before the dashboard was built. Future improvements could include replacing the KY-039 with a MAX30102 optical sensor for calibrated SpO₂ and BPM measurement, integrating an MPU-6050 IMU for accelerometer-based step detection, and extending the web application with session history charts and data export.

8 Sources and References

- [1] Github, *Github Repo Gr_B*, https://github.com/MoAbdoHSHL/Adv.Embedded-Systems-Gr_B.
- [2] Arduino, *Arduino Nano ESP32 Documentation*, <https://docs.arduino.cc/hardware/nano-esp32>, 2024.
- [3] MQTT.org, *MQTT: The Standard for IoT Messaging*, <https://mqtt.org>, 2024.
- [4] Eclipse Foundation, *Eclipse Mosquitto: An Open Source MQTT Broker*, <https://mosquitto.org>, 2024.
- [5] Raspberry Pi Foundation, *Raspberry Pi 3 Model B Documentation*, <https://www.raspberrypi.com>, 2024.
- [6] Joy-IT, *KY-039 Heartbeat Sensor — Sensorkit X40*, <https://sensorkit.joy-it.net/en/sensors/ky-039>, 2024.
- [7] Joy-IT, *KY-017 Tilt Switch — Sensorkit X40*, <https://sensorkit.joy-it.net/en/sensors/ky-017>, 2024.
- [8] Joy-IT, *KY-016 RGB 5mm LED — Sensorkit X40*, <https://sensorkit.joy-it.net/en/sensors/ky-016>, 2024.
- [9] Joy-IT, *KY-012 Active Buzzer Module — Sensorkit X40*, <https://sensorkit.joy-it.net/en/sensors/ky-012>, 2024.